Hiding SNI and destinations from the ISP

A generic guide to shrinking the leak surface netmon reveals. Run netmon first to see your own baseline, then apply and re-verify. The typical leak surface on an unmitigated host is: TLS SNI of every connection, destination IPs, and (if you run a mesh VPN like Tailscale) its coordination/STUN chatter. Encrypted DNS (DoT/DoH) closes the DNS channel; confirm yours with a capture showing no plaintext `udp/53` on the physical interface.

> A host-specific version of this file, with measured captures for one machine, may be kept locally as `MITIGATIONS.local.md` (gitignored). Keep measurements and "this host runs X" details out of the tracked copy — they fingerprint your setup.

Option 1 — Encrypted Client Hello (free, partial)

ECH is RFC 9849 (March 2026). The browser sends a cleartext outer ClientHello with a generic cover SNI (Cloudflare uses `cloudflare-ech.com` for everyone) and the real hostname rides encrypted inside.

What it needs:

- **Encrypted DNS** — ECH configs arrive in `HTTPS` DNS records; with plaintext DNS the hostname leaks in the query anyway.
- **Firefox** — on by default since v119; since v129 it fetches HTTPS records through the OS resolver, so it works with system-level DoT **without enabling browser DoH** (Fx129 notes). Caveat: Firefox doesn't verify the OS resolver is actually encrypted.
- **Chromium/Chrome** — only activates ECH when the browser's own "Use secure DNS" (DoH) toggle is on. System DoT does not count.
- **Verify**: visit `https://tls-ech.dev/` and `https://defo.ie/ech-check.php` with netmon running — the SNI in `tls.jsonl` should read `public.tls-ech.dev` / `cover.defo.ie`, not the real name.

Hard limits (why ECH alone is not enough):

- **Per-site**: only works when the site publishes an `ech=` key in its HTTPS record. Roughly 20% of the top 1M sites publish ECH configs (Scott Helme census, June 2026), nearly all via CDNs — and coverage is uneven even within a single CDN's zones.
- **IP still visible**: ECH's value is k-anonymity behind shared CDN IPs. For single-tenant IPs the ISP maps IP→site regardless.
- **CLI tools are naked**: many distributions' curl has no ECH compiled in — scripts, package managers, and apps keep leaking SNI.
- ECH use is itself fingerprintable; hostile networks can strip/block it (observed in the wild), and browsers have been seen retrying without ECH — which puts the real SNI back on the wire.

Option 2 — Route everything through a tunnel (robust)

A full-tunnel exit collapses the whole leak surface to "encrypted blob to one endpoint". If a mesh VPN (e.g. Tailscale) is already running:

- **Self-hosted exit node** (a VPS): on the VPS `sudo tailscale set --advertise-exit-node` (plus IP forwarding), then on the client `sudo tailscale set --exit-node=<node>`. DNS follows the exit node by default. The VPS provider now sees the traffic instead of the ISP — a trust transfer, not elimination. A home exit node only helps when away from home.
- **Mullvad exit nodes** (Tailscale add-on, ~$5/month for 5 devices): Mullvad's fleet as exit nodes, no separate app. Tailscale knows who you are but Mullvad doesn't ("conditional anonymity"). Enable in the Tailscale admin console, then `tailscale set --exit-node=<mullvad-node>`.
- **Plain WireGuard / commercial VPN** works equally, with caveats:
  - `wg-quick` alone handles none of the leak classes by default: DNS only if the config has a `DNS=` line (and systemd-resolved honors it), IPv6 only if `AllowedIPs` includes `::/0` (many provider configs omit it → native IPv6 bypasses the tunnel), and there is no boot/reconnect kill switch at all.
  - The Mullvad app closes all three: always-on nftables kill switch (cannot be disabled), DNS forced through the tunnel, IPv4+IPv6 both routed, early-boot systemd blocking unit. Mullvad's no-log claim has strong public evidence: recurring third-party audits (X41 2024, Cure53 2024) and a 2023 Swedish police raid that left empty-handed.
  - **Do not run another VPN daemon alongside `tailscaled`** — Tailscale documents the conflict explicitly (firewall rules,

CGNAT 100.64/10 overlap, DNS fights) and names Mullvad specifically; the Mullvad exit-node add-on is Tailscale's recommended way to combine them.
    - Reconcile DNS deliberately: a host that runs systemd-resolved→DoT will fight a VPN app's forced DNS. Simplest correct config: let the VPN's DNS win while the tunnel is up (or `tailscale set --accept-dns=false` in tailnet setups with DNS breakage).
    - The ISP can still identify WireGuard as a protocol (fixed handshake sizes, UDP timing) even though contents are opaque. If "ISP shouldn't know I use a VPN at all" is part of the threat model, that needs obfuscated transports (Mullvad ships Shadowsocks-style bridges), not plain WireGuard.

What the ISP still sees with an exit node active:

- WireGuard UDP to one peer (or, if UDP is blocked, TLS to `derp*.tailscale.com` on tcp/443)
- Control plane: `controlplane.tailscale.com`, plus STUN/DERP probe chatter
- Client log uploads to `log.tailscale.com` — disable with `TS_NO_LOGS_NO_SUPPORT=true` in `/etc/default/tailscaled` (forfeits Tailscale support)
- Traffic volume and timing — no tunnel hides that

So the ISP learns "uses a VPN, this much traffic, at these times" and nothing else. This covers curl/apt/every app, not just browsers.

Option 3 — Upstream VPN on the firewall (pfSense)

Full-tunnel VPN on pfSense, devices behind it untouched. No daemon conflicts on the device — DoT rides the tunnel like any other flow, and the trust splits: ISP sees only the tunnel, the VPN provider can't read the DoT stream, the DNS resolver sees queries from the VPN exit IP. Defaults to harden (verified against docs.netgate.com):

- Policy routing **fails open** when the VPN gateway drops — traffic falls back to WAN. Enable System > Advanced > Miscellaneous > "Do not create rules when gateway is down", plus explicit LAN→WAN block rules so failure means "no internet", not "via ISP".
- IPv6 from the ISP bypasses an IPv4-only tunnel entirely — tunnel it or block IPv6 egress on WAN.
- DoT (tcp/853) bypasses port-53 redirect rules by design — protection must come from full-tunnel policy routing of the VLAN, not DNS redirection.

- pfSense's unbound sources queries per routing table (leaky): set Services > DNS Resolver > Outgoing Network Interfaces to the VPN interface.
- Verify at the firewall, not the device: Diagnostics > Packet Capture on WAN (or netmon on a WAN mirror port) — anything to port 53/853 or a real destination IP is a leak.

Verification protocol (netmon)

1. `uv run netmon.py -q`, browse normally for a few minutes, stop.
2. Before mitigation: `tls.jsonl` reads like browsing history.
3. With ECH working: ECH-enabled sites appear as cover names only.
4. With an exit node or VPN: `tls.jsonl` should contain **no** real site SNI; `flows.jsonl` internet-scope should reduce to the tunnel endpoint plus (if a mesh VPN runs) its infrastructure. Specifically check for these leak signatures on the physical interface:
   - any `dns`/`dot` flows (udp/53, tcp/853) to a non-tunnel address — DNS escaping the tunnel
   - any IPv6 flow to a global address — the classic `::/0`-missing leak
   - any real SNI — an app bound to the physical interface or a kill-switch gap
   - `-i <physical-iface>` narrows the capture to exactly the ISP's viewpoint

Choosing an approach

1. Keep encrypted DNS (DoT/DoH) if you have it — it's the baseline.
2. Confirm Firefox ECH once via `tls-ech.dev` with netmon watching. Free win for CDN-fronted sites, no action needed.
3. For actual "ISP sees nothing" coverage, add an exit node: the Mullvad add-on if paying ~$5/mo is fine, otherwise a cheap VPS as a self-hosted exit. Verify with the protocol above.
4. Accept the residual: exit-node metadata (volume/timing, "uses a VPN") is visible; only Tor-class tooling addresses traffic correlation, at a very different cost.